Lab 7: Fuzzing for Input Validation Bugs

Atharva Ashish Vyas | PES1UG24CS093
Application: customs duty (same application as Lab 6) — processor.py, test_processor.py

Summary

Hypothesis fuzzing was applied to **duty_rate**, **total_duty** and **clearance_channel**. The property tested: for any input a function returns a sensible result or raises **ValueError**; any other exception, or a silently wrong result, is a bug. On the original code 8 of 16 tests fail; after the fixes all 16 pass.

Bugs found

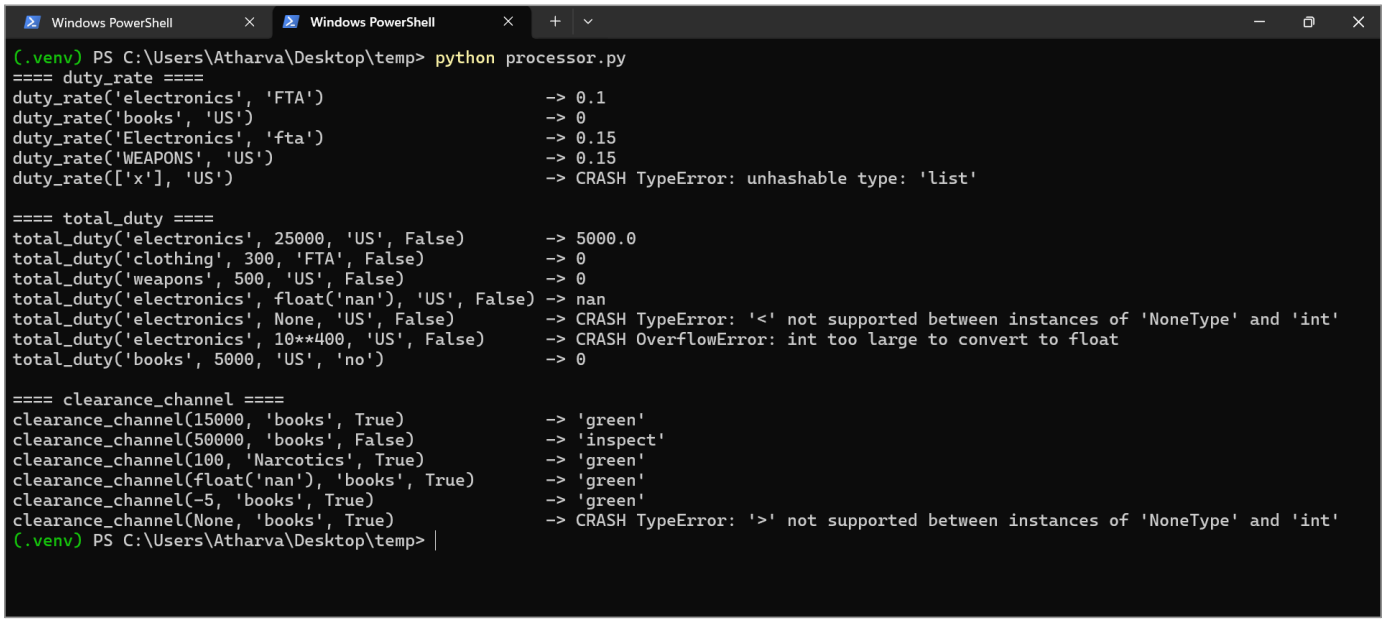
1. **Crashes on bad input**: None / list / non-numeric values raise TypeError (unhashable list, None compared with int); 10**400 raises OverflowError.
2. **nan accepted**: total_duty returns nan; clearance_channel returns 'green' for nan and for negative values.
3. **Prohibited-item bypass**: 'Weapons' / 'NARCOTICS' are not matched (case-sensitive check), and total_duty returned 0 for prohibited items below the exemption thresholds before the prohibited check ran.
4. **Case-sensitive category / origin**: 'Electronics' got the default 15% instead of 20%; 'fta' was not treated as FTA.
5. **Truthy flags**: is_gift='no' was treated as a gift.

Fix

Input validators check and normalise every argument (string category/origin, finite non-negative number that fits in a float, real bool flags). Categories and origins are compared case-insensitively, and prohibited categories are rejected before any exemption applies. All invalid input raises ValueError.

1. Execution of the original application

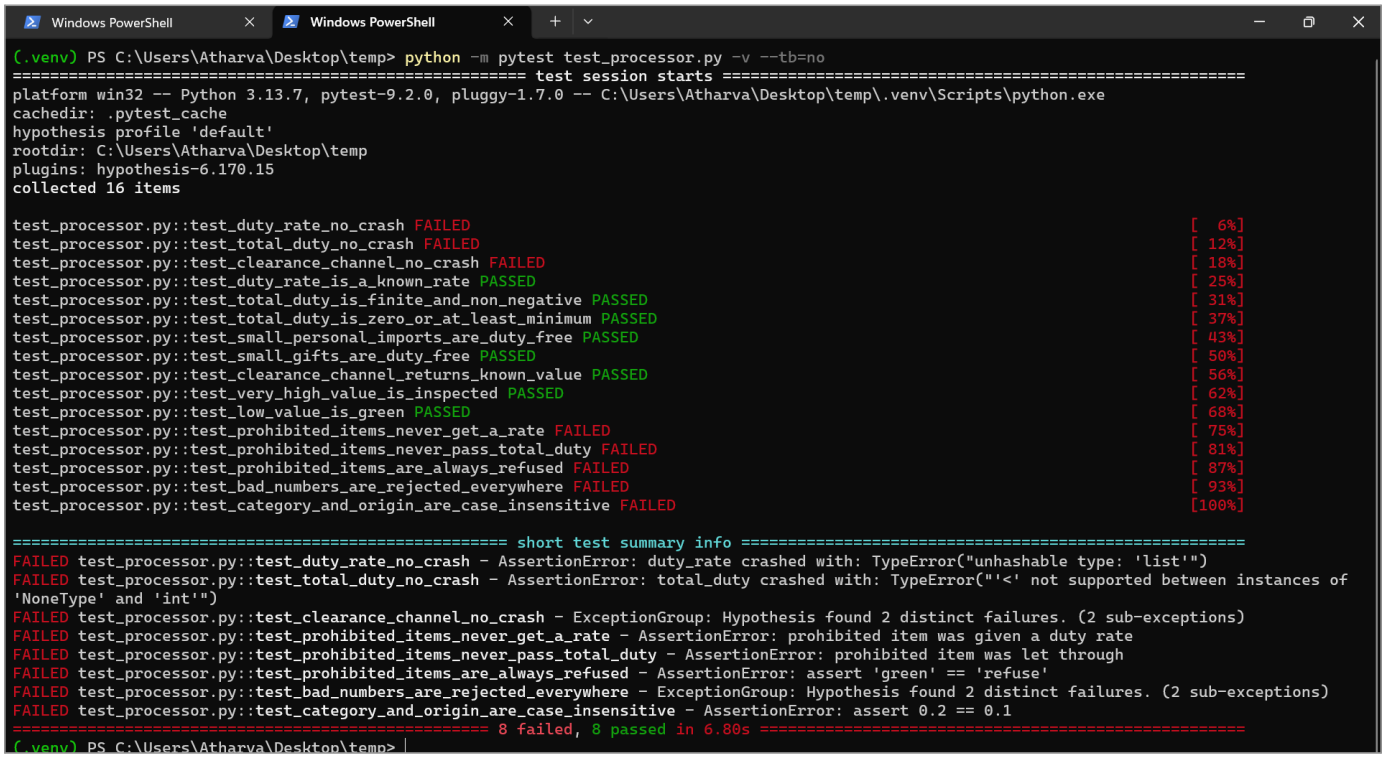
`python processor.py`



Crashes (TypeError, OverflowError) and silent wrong results: 'WEAPONS' gets a rate, weapons pass with duty 0, nan is returned, and 'Narcotics', nan and -5 are all cleared 'green'.

2. Failing fuzz tests (original application)

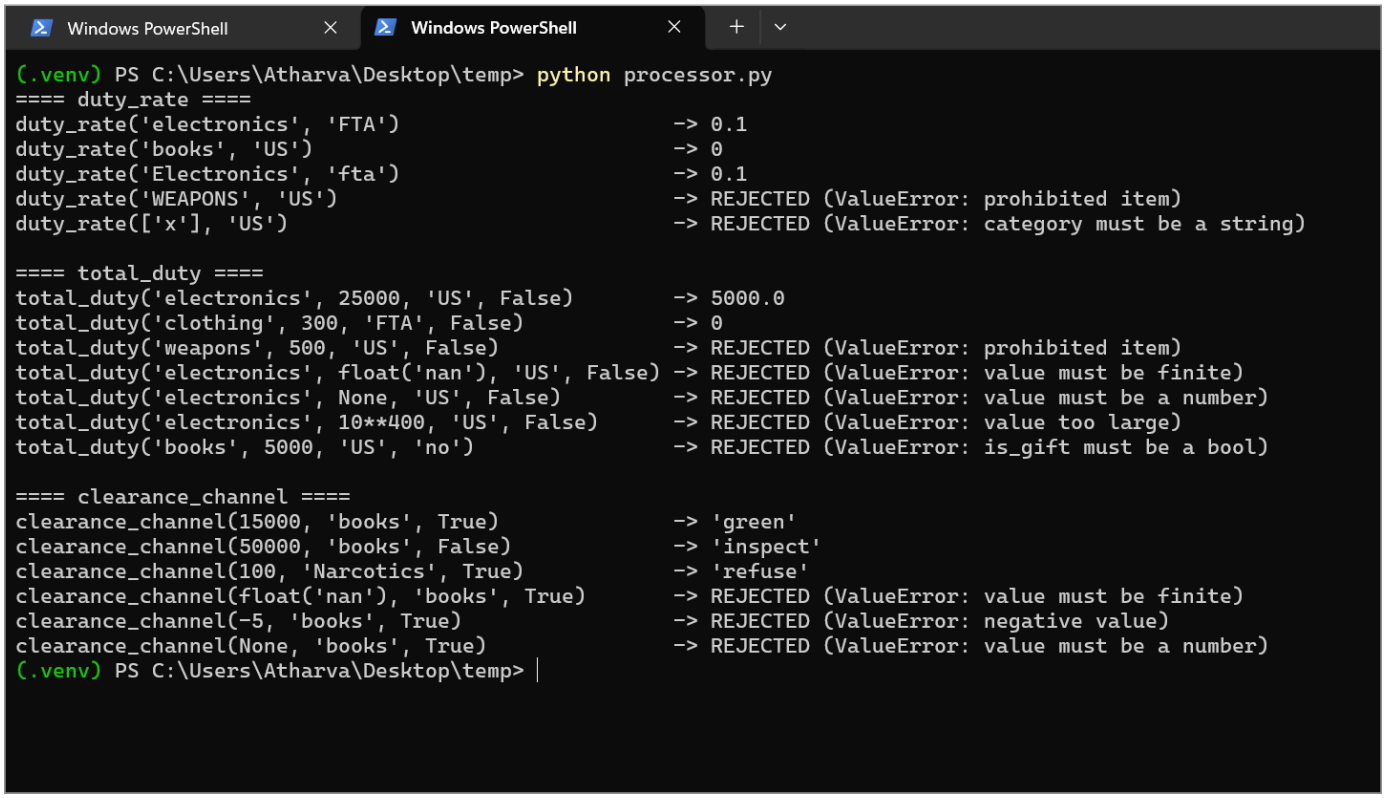
```
python -m pytest test_processor.py -v --tb=no
```



8 failed, 8 passed. The no-crash tests fail with TypeError; the security tests show prohibited items being given a rate / let through / not refused, and case-insensitivity and nan handling failing.

3. Execution of the corrected application

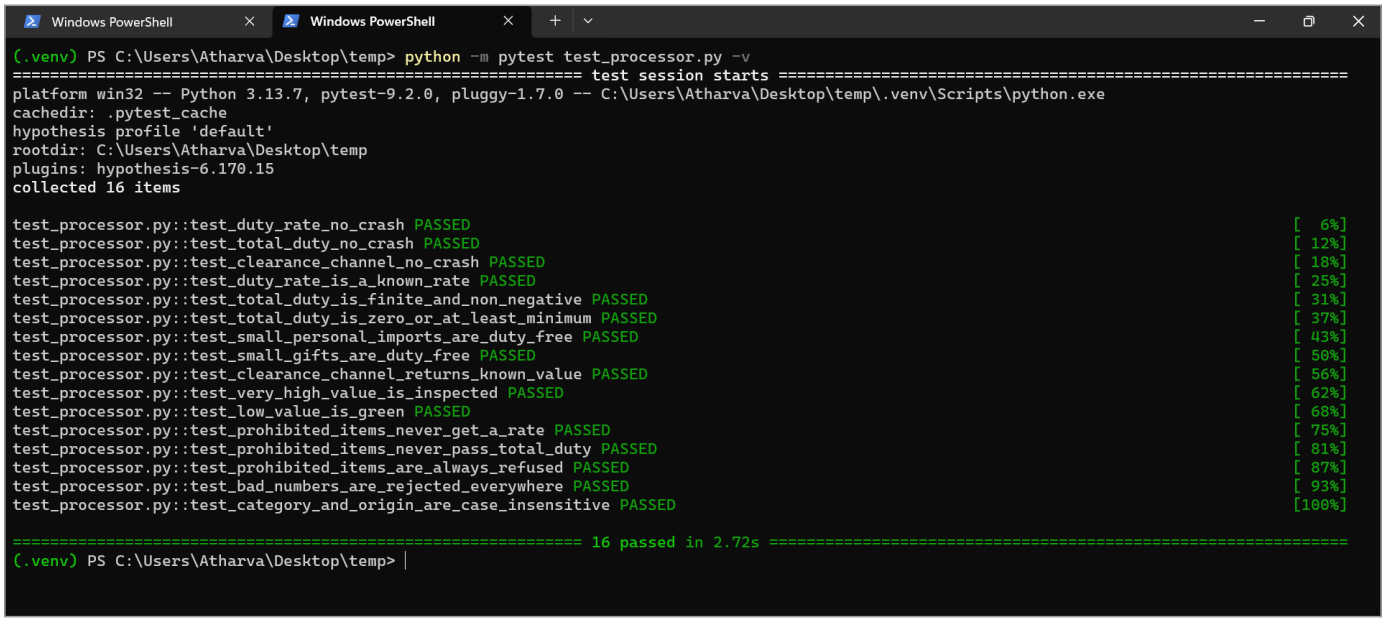
`python processor.py`



The same inputs are now rejected with a clear ValueError, 'Electronics'/'fta' gets the correct 0.1 rate, and 'Narcotics' is refused; valid inputs still give the correct results.

4. Final test run (corrected application)

```
python -m pytest test_processor.py -v
```



All 16 tests pass.

Source code: processor.py (fixed)

```
import math

PROHIBITED = {"weapons", "narcotics"}

# Contract: every function validates its inputs and signals ANY invalid input
# with ValueError (never TypeError / AttributeError / OverflowError, and never
# a silent nan/inf result).


def _clean_category(category):
    if not isinstance(category, str):
        raise ValueError("category must be a string")
    category = category.strip().lower()      # 'WEAPONS ' must match 'weapons'
    if not category:
        raise ValueError("category must not be empty")
    return category


def _clean_origin(origin):
    if not isinstance(origin, str):
        raise ValueError("origin must be a string")
    return origin.strip().upper()            # 'fta' == 'FTA'


def _clean_value(value):
    if isinstance(value, bool) or not isinstance(value, (int, float)):
        raise ValueError("value must be a number")
    try:
        as_float = float(value)              # huge ints raise OverflowError
    except OverflowError:
        raise ValueError("value too large")
    if not math.isfinite(as_float):          # nan / inf
        raise ValueError("value must be finite")
    if as_float < 0:
        raise ValueError("negative value")
    return value


def _clean_flag(flag, name):
    if not isinstance(flag, bool):
        raise ValueError(f"{name} must be a bool")
    return flag


def duty_rate(category, origin):
    category = _clean_category(category)
    origin = _clean_origin(origin)
    if category in PROHIBITED:
        raise ValueError("prohibited item")
    if category == "electronics":
        if origin == "FTA":
            return 0.10
        return 0.20
    if category == "clothing":
        if origin == "FTA":
            return 0.05
        return 0.12
    if category == "books":
        return 0
    return 0.15


def total_duty(category, value, origin, is_gift):
    category = _clean_category(category)
    value = _clean_value(value)
    origin = _clean_origin(origin)
    is_gift = _clean_flag(is_gift, "is_gift")
    # Prohibited items are rejected BEFORE any exemption can return 0.
    if category in PROHIBITED:
        raise ValueError("prohibited item")
    if is_gift:
        if value <= 5000:
            return 0
        value -= 5000
    elif value <= 1000:
        return 0
    duty = value * duty_rate(category, origin)
    if 0 < duty < 50:
        duty = 50
    return duty


def clearance_channel(value, category, has_documents):
    value = _clean_value(value)
```

```
    category = _clean_category(category)
    has_documents = _clean_flag(has_documents, "has_documents")
    if category in PROHIBITED:
        return "refuse"
    if value > 100000:
        return "inspect"
    if value > 20000 and not has_documents:
        return "inspect"
    return "green"


def _show(label, fn, *args):
    try:
        print(f"{label:<52} -> {fn(*args)!r}")
    except ValueError as e:
        print(f"{label:<52} -> REJECTED (ValueError: {e})")
    except Exception as e:
        print(f"{label:<52} -> CRASH {type(e).__name__}: {e}")


def main():
    print("==== duty_rate ====")
    _show("duty_rate('electronics', 'FTA')", duty_rate, "electronics", "FTA")
    _show("duty_rate('books', 'US')", duty_rate, "books", "US")
    _show("duty_rate('Electronics', 'fta')", duty_rate, "Electronics", "fta")
    _show("duty_rate('WEAPONS', 'US')", duty_rate, "WEAPONS", "US")
    _show("duty_rate(['x'], 'US')", duty_rate, ["x"], "US")

    print("\n==== total_duty ====")
    _show("total_duty('electronics', 25000, 'US', False)", total_duty, "electronics", 25000, "US", False)
    _show("total_duty('clothing', 300, 'FTA', False)", total_duty, "clothing", 300, "FTA", False)
    _show("total_duty('weapons', 500, 'US', False)", total_duty, "weapons", 500, "US", False)
    _show("total_duty('electronics', float('nan'), 'US', False)", total_duty, "electronics", float("nan"), "US", False)
    _show("total_duty('electronics', None, 'US', False)", total_duty, "electronics", None, "US", False)
    _show("total_duty('electronics', 10**400, 'US', False)", total_duty, "electronics", 10**400, "US", False)
    _show("total_duty('books', 5000, 'US', 'no')", total_duty, "books", 5000, "US", "no")

    print("\n==== clearance_channel ====")
    _show("clearance_channel(15000, 'books', True)", clearance_channel, 15000, "books", True)
    _show("clearance_channel(50000, 'books', False)", clearance_channel, 50000, "books", False)
    _show("clearance_channel(100, 'Narcotics', True)", clearance_channel, 100, "Narcotics", True)
    _show("clearance_channel(float('nan'), 'books', True)", clearance_channel, float("nan"), "books", True)
    _show("clearance_channel(-5, 'books', True)", clearance_channel, -5, "books", True)
    _show("clearance_channel(None, 'books', True)", clearance_channel, None, "books", True)


if __name__ == "__main__":
    main()
```

Source code: test_processor.py

```
"""Hypothesis fuzz tests for processor.py (customs duty application).

Contract being tested: for ANY input a function either returns a sensible
result or raises ValueError. Any other exception (TypeError, OverflowError,
AttributeError ...) or a silently wrong result (nan, wrong channel,
prohibited item let through) is a bug.
"""
import math

from hypothesis import assume, given, strategies as st

from processor import PROHIBITED, clearance_channel, duty_rate, total_duty

# ---------- strategies ----------
# Anything at all: wrong types, nan/inf, huge ints, unhashable lists ...
anything = (
    st.none()
    | st.text()
    | st.integers()
    | st.floats()                      # includes nan, inf, -inf
    | st.booleans()
    | st.binary()
    | st.lists(st.integers())
)
# Valid building blocks
valid_value = st.floats(min_value=0, max_value=1e12, allow_nan=False, allow_infinity=False)
valid_category = st.text(min_size=1).filter(lambda s: s.strip())
known_category = st.sampled_from(["electronics", "clothing", "books", "toys", "Electronics", " CLOTHING "])
prohibited_category = st.sampled_from(
    ["weapons", "narcotics", "Weapons", "NARCOTICS", "  weapons  ", "NarCotics"]
)
origin = st.text() | st.sampled_from(["FTA", "fta", " Fta "])


def norm(c):
    return c.strip().lower()


# ---------- 1. No-crash fuzz tests (one per function) ----------

@given(anything, anything)
def test_duty_rate_no_crash(category, org):
    try:
        duty_rate(category, org)
    except ValueError:
        pass
    except Exception as e:
        assert False, f"duty_rate crashed with: {e!r}"


@given(anything, anything, anything, anything)
def test_total_duty_no_crash(category, value, org, gift):
    try:
        total_duty(category, value, org, gift)
    except ValueError:
        pass
    except Exception as e:
        assert False, f"total_duty crashed with: {e!r}"


@given(anything, anything, anything)
def test_clearance_channel_no_crash(value, category, docs):
    try:
        clearance_channel(value, category, docs)
    except ValueError:
        pass
    except Exception as e:
        assert False, f"clearance_channel crashed with: {e!r}"


# ---------- 2. Result-correctness properties ----------

@given(valid_category, origin)
def test_duty_rate_is_a_known_rate(category, org):
    assume(norm(category) not in PROHIBITED)
    assert duty_rate(category, org) in {0, 0.05, 0.10, 0.12, 0.15, 0.20}


@given(valid_category, valid_value, origin, st.booleans())
def test_total_duty_is_finite_and_non_negative(category, value, org, gift):
    assume(norm(category) not in PROHIBITED)
    d = total_duty(category, value, org, gift)
    assert math.isfinite(d) and d >= 0
    assert d <= max(50, value * 0.20)          # never above top rate / minimum duty
```

```


@given(valid_category, valid_value, origin, st.booleans())
def test_total_duty_is_zero_or_at_least_minimum(category, value, org, gift):
    assume(norm(category) not in PROHIBITED)
    d = total_duty(category, value, org, gift)
    assert d == 0 or d >= 50


@given(known_category, st.floats(min_value=0, max_value=1000, allow_nan=False), origin)
def test_small_personal_imports_are_duty_free(category, value, org):
    assert total_duty(category, value, org, False) == 0


@given(known_category, st.floats(min_value=0, max_value=5000, allow_nan=False), origin)
def test_small_gifts_are_duty_free(category, value, org):
    assert total_duty(category, value, org, True) == 0


@given(valid_value, valid_category, st.booleans())
def test_clearance_channel_returns_known_value(value, category, docs):
    assert clearance_channel(value, category, docs) in {"refuse", "inspect", "green"}


@given(st.floats(min_value=100000.01, max_value=1e12), known_category, st.booleans())
def test_very_high_value_is_inspected(value, category, docs):
    assert clearance_channel(value, category, docs) == "inspect"


@given(st.floats(min_value=0, max_value=20000, allow_nan=False), known_category, st.booleans())
def test_low_value_is_green(value, category, docs):
    assert clearance_channel(value, category, docs) == "green"


# ---------- 3. Security-style properties (found by fuzzing) ----------

@given(prohibited_category, origin)
def test_prohibited_items_never_get_a_rate(category, org):
    try:
        duty_rate(category, org)
        assert False, "prohibited item was given a duty rate"
    except ValueError:
        pass


@given(prohibited_category, valid_value, origin, st.booleans())
def test_prohibited_items_never_pass_total_duty(category, value, org, gift):
    # even tiny values / gifts must not slip through the duty-free exemptions
    try:
        total_duty(category, value, org, gift)
        assert False, "prohibited item was let through"
    except ValueError:
        pass


@given(valid_value, prohibited_category, st.booleans())
def test_prohibited_items_are_always_refused(value, category, docs):
    assert clearance_channel(value, category, docs) == "refuse"


@given(st.sampled_from([float("nan"), float("inf"), float("-inf"), -1, -0.01, 10**400]))
def test_bad_numbers_are_rejected_everywhere(bad):
    for call in (
        lambda: total_duty("electronics", bad, "US", False),
        lambda: clearance_channel(bad, "books", True),
    ):
        try:
            call()
            assert False, f"{bad!r} was accepted"
        except ValueError:
            pass


@given(known_category, origin)
def test_category_and_origin_are_case_insensitive(category, org):
    assert duty_rate(category, org) == duty_rate(norm(category), org.strip().upper())
```